

Local development

A `docker-compose.yml` at the repo root brings up this framework's own services (`webhook-listener`, `dispatch-worker`) plus enough of Temporal and AWS to actually exercise the whole dispatch loop — a story entering `In Progress` can start a real Temporal workflow that runs a real ECS `RunTask` call and launches a real `specialist-runner` container.

What's real, what's local:

Piece	Local dev
Linear, GitHub, Anthropic	Real, external. Nothing here emulates any of the three — same credentials, same hosted APIs as production.
Temporal	Local. <code>temporalio/auto-setup</code> + Postgres + <code>temporal-ui</code> , not Temporal Cloud — no TLS, no namespace API key.
AWS (<code>ecs:RunTask/DescribeTasks</code>)	Local , via <code>LocalStack</code> . <code>dispatch-specialist.ts</code> 's and <code>await-specialist-task.ts</code> 's code is unmodified — the AWS SDK already respects <code>AWS_ENDPOINT_URL</code> natively, confirmed by reading the installed package directly.

This is a first pass, not a claim that it mirrors production's real VPC/IAM shape — the `LocalStack` bootstrap creates the smallest possible dummy VPC/subnet/security-group, not `infrastructure/`'s real `network` stack.

Prerequisites

- Docker Desktop (or an equivalent Docker engine) running.
- A **free LocalStack account and auth token** — as of LocalStack's 2026-03 licensing change, the separately-distributed Community image was discontinued; the single remaining image now requires an account + token even for the permanent free (non-commercial) tier. Sign up at app.localstack.cloud and generate one — see Bring-up order below for where it goes.
- A real Linear sandbox workspace, a real GitHub org/repo you can write branches and PRs to, and real API keys for Linear, GitHub, and Anthropic.
- [cloudflared](https://cloudflared.com) (or any tunnel tool) — compose doesn't change how Linear reaches your machine; see [webhook-listener/README.md](#)'s existing local-dev section for the same step this always needed.

Bring-up order

- 1 **Copy** `docker-compose.override.yml.example` to `docker-compose.override.yml` and fill in your **real secret values**. Every env var each service needs is already listed in `docker-compose.yml` itself — one file to see the whole shape — with real working defaults for everything non-secret (Temporal connection info for this stack, AWS/LocalStack placeholder creds) and empty placeholders for the handful that are actually secret (Anthropic/GitHub/Linear keys, the LocalStack token). `docker compose` auto-merges `docker-compose.override.yml` if present (gitignored — never commit your filled-in copy); it only needs to name the specific keys it's overriding, since Compose merges `environment:` maps key-by-key.

One non-secret value must be overridden too: `FRAMEWORK_REPO` on `localstack-bootstrap`. It names the framework repo (`org/name`) the specialist reads its agent definition and skills from, and `docker-compose.yml` ships a placeholder. Set `FRAMEWORK_REF` alongside it (default `main`) to a feature branch when testing framework changes locally.

To exercise revision rounds, also set `REVIEWER_EMAIL_TO_GITHUB_LOGIN` on `dispatch-worker`, a JSON object mapping your Linear email to your GitHub login (e.g. `{"you@example.com":"your-github-login"}`). Without an entry for whoever moves the story, `dispatch` still works, but no reviewer is requested and a "request changes" review does nothing.

The per-package `.env.example` files (`webhook-listener/`, `dispatch-worker/`, `specialist-runner/`) are unrelated to this compose stack — they're for running one package standalone outside Docker (each README's own "Run it locally" section) or, for `specialist-runner`, testing the image by hand. Nothing here reads them.

- 1 **Build the specialist-runner image once, by hand** — it's not a compose service (see "Why specialist-runner isn't a compose service" below):

```
``bash docker build -f specialist-runner/Dockerfile -t specialist-runner:local
specialist-runner ``
```

- 1 **Bring up the stack:**

```
``bash docker compose up --build ``
```

Watch for `localstack-bootstrap` to exit 0 (it's a one-shot job — `docker compose ps` will show it `Exited (0)`) before `dispatch-worker` starts; compose's own `depends_on: condition: service_completed_successfully` already enforces this ordering.

- 1 **Tunnel a real Linear webhook in**, same as `webhook-listener/README.md` already documents without compose:

```
``bash cloudflared tunnel --url http://localhost:8787 ``
```

Put the resulting URL (`https://.../webhooks/linear`) into your Linear sandbox's webhook settings (Settings → API → Webhooks).

Triggering and watching a dispatch

Before a status move can dispatch anything, the sandbox tracker and repo need what a real engagement has:

- **A `Surfaces` document on the project**, with a record for each surface the story is labelled with (repo, ref, and optionally path, conventions file, mandatory skills). Without a matching record, dispatch fails and the story goes back to Todo with a comment.
- **A story carrying a `surface:<name>` label** that matches a record. The lane ignores a story with no `surface:` label.
- **The epic's branch in the target repo**, named as the tracker names it and cut from a BRD branch. The app creates the story branch; it does not create the epic branch.
- **A `CONVENTIONS.md`** at the surface root on that ref. The specialist stops without one.
- **A `Blocking dependencies` section** in the story's description ("No blocking dependencies." is fine). Every story it names must be Done.

Then move the story to the status named `In Progress` in your Linear sandbox; that exact name is what the lane matches. Watch it happen:

- **Temporal UI** — `http://localhost:8080` — the workflow execution, its activities, and (once one's dispatched) the heartbeat/status detail `await-pull-request-outcome.ts` reports.
- `docker compose logs -f webhook-listener dispatch-worker` — the same structured logs described in each package's own README.
- **A real container actually launching** — `docker ps` should show a `specialist-runner:local` container appear once `dispatch-specialist.ts`'s `RunTask` call lands (LocalStack's Fargate emulation runs it via the host Docker engine, the same shape as ECS in production).

Why `specialist-runner` isn't a compose service

It's ephemeral by design — one container per dispatch, launched by ECS (in production) or LocalStack's Fargate emulation (here), never a long-running process. LocalStack finds `specialist-runner:local` in the **local Docker image cache** when it runs the task — it does not rebuild it. After any code change under `specialist-runner/`, rebuild the image (step 2 above) before triggering another dispatch; a stale image is a real, easy-to-hit local-dev footgun worth knowing about up front, not a bug in the bootstrap or the workflow.

How `dispatch-worker` gets the LocalStack values

`localstack-bootstrap` writes the `SPECIALIST_*` values (cluster, task definition, subnet, security group) to `local/localstack.env`. `docker-compose.yml`'s `env_file:` resolves at container *creation* time, not process-start time, so `dispatch-worker` could never see those values within one `up` through `env_file` alone. Instead the file is bind-mounted and read inside the container at process start (`dispatch-worker/src/local-env-file.ts`).

What this does NOT do

- No live-reload — each service rebuilds from its existing Dockerfile on `docker compose up --build`, same as production. No volume-mounted source, no `nodemon`.
- No mocked Linear/GitHub/Anthropic — a real sandbox workspace, real repo, and real API keys are required to drive anything past `webhook-listener` accepting a webhook.

- Does not exercise [infrastructure/](#)'s real CDKTN stacks — the LocalStack bootstrap ([scripts/localstack-bootstrap.sh](#)) is a small, standalone script, not those stacks pointed at a different endpoint.